
WHERE TO SPARSIFY A TRANSFORMER: MATCHED INTERVENTIONS IN ARCHITECTURE AND OPTIMIZATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation sparsity offers a route to efficient inference, but its value depends on where zeros occur and how training accommodates them. We study these choices through matched interventions in transformer architecture and optimization: post-hoc clipping, feed-forward activation pressure, broader gating, and pressure at the gated sites. A detailed 14M-parameter study and selected 70M experiments compare validation loss with model-wide activation sparsity opportunity, the fraction of counted multiplications with an exact-zero operand. At 14M, expanding pressure from the feed-forward hidden state to four sites increases loss at every tested threshold. Adding gates to attention queries, keys, and values changes the response to pressure: at the largest threshold, seven-site pressure adds 12.10 percentage points of opportunity with a 0.127 loss increase, compared with 2.50 points and 0.378 for four-site pressure. Trained interventions extend the evaluated uniform-clipping frontier at both sizes, with benefits depending on the allowed loss increase. Operation-level counts explain the contribution of attention sparsity and the effect of architecture on cross-size comparisons. A correctness-checked 70M implementation study yields slower sparse execution in all tested settings. These single-seed results identify site selection and optimization as coupled design choices, and motivate interventions that also control the structure required for efficient execution.

1 INTRODUCTION

Reducing inference cost requires models whose computations can be executed efficiently. Activation sparsity is one approach: when an input is zero, its contribution to a multiplication is known, creating an opportunity to avoid work. The useful opportunity depends on both the number of zeros and the operations they enter. A sparse feed-forward hidden state affects one projection; sparse inputs to attention and feed-forward projections affect other matrix products; sparse queries, keys, and values can also affect the attention products. Training determines how much of this sparsity a model can accommodate at a given prediction quality.

Existing work approaches this problem from several directions. Training-free methods remove small activations from a fixed model (Liu et al., 2025; Lee et al., 2024). Training interventions introduce or encourage sparsity, often through feed-forward activations (Mirzadeh et al., 2023; Song et al., 2024; Zhang et al., 2024). Broader approaches already include all linear inputs, structured masks, and end-to-end execution (Wang et al., 2024; You et al., 2025; Cetin et al., 2026). These advances make a more specific scientific question useful: *how does the effect of sparsity pressure change when the set of gated activations changes?* Answering it requires matched architecture and optimization comparisons, together with a common accounting of the operations each intervention can affect.

We investigate this question with an intervention ladder containing controlled branches. At 14M parameters, we first vary pressure on the feed-forward hidden state. We then hold a four-site gate configuration fixed while expanding the pressure sites. Finally, we add sign-preserving gates to attention queries, keys, and values and compare training with and without pressure within each gate configuration. These comparisons share initialization, data order, and training schedule. Each threshold gives a matched contrast; the five thresholds characterize a dose response under one seed.

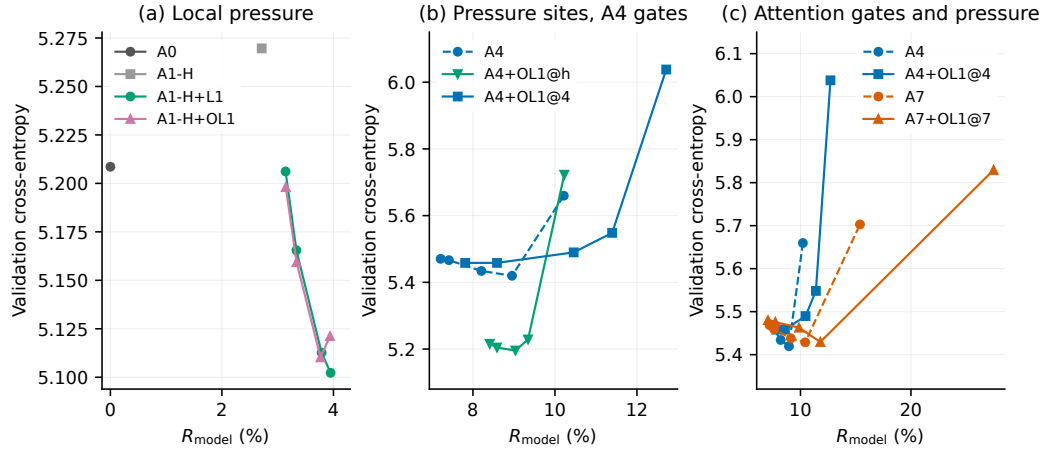


Figure 1: Matched 14M intervention branches. Left: pressure on the feed-forward hidden state under a ReLU activation. Middle: four-site gating with pressure applied to zero, one, or four sites. Right: four- and seven-site gates, with and without pressure at their respective sites. Each line joins the tested pressure coefficients or gate thresholds. Lower loss and higher model-wide opportunity are preferred. Complete values and the negative pressure-site comparisons appear in Appendix B.

The central exploratory finding is that broader pressure has a different effect depending on where gates are placed. With four gated sites, spreading pressure beyond the feed-forward hidden state raises loss throughout the tested grid. With additional query, key, and value gates, pressure produces a substantially larger increase in model-wide opportunity at the largest threshold, with a smaller loss increase. The pressured four- and seven-site recipes change both the gate set and the pressure set. Their comparison therefore identifies a promising combined intervention and motivates a fixed-pressure follow-up to separate these two contributions.

We evaluate opportunity as the fraction of scalar multiplications with an exact-zero operand across the transformer blocks and dense output head. This operation-weighted measure supports the intervention comparisons: different gate sets affect different products, and the output head changes the model-wide fraction across architectures. It complements validation loss and measured latency.

Our contribution is an empirical account of how gate location and optimization jointly shape the quality–sparsity frontier. It combines matched 14M comparisons, a selected 70M extension of the pressured recipes, and operation-level explanations of their gains and limits. The trained conditions extend the evaluated uniform-clipping frontier, although clipping remains competitive and is stronger under tight quality allowances at 70M. We also report a negative 70M execution result and retain a fixed-budget 410M study in the appendix. Together, these observations distinguish an improvement in learned sparsity from the additional structure and implementation work required to turn it into faster inference.

2 RELATED WORK

Training-free and trained sparsity. TEAL removes low-magnitude activations and allocates sparsity across a model using a blockwise procedure (Liu et al., 2025). CATS studies activation thresholding (Lee et al., 2024). ReLU-based models, progressive regularization, and sparse pretraining establish that training can accommodate substantial feed-forward sparsity (Mirzadeh et al., 2023; Song et al., 2024; Zhang et al., 2024). The Sparsing Law studies how activation sparsity relates to model size, data, and quality (Luo et al., 2025). Our uniform, calibrated clipping baseline provides a fixed-model reference for the same checkpoints and validation coverage. TEAL’s optimized allocation and the full training recipes of these methods remain important comparisons.

Sparsity across operations. Q-Sparse is a close precedent: it applies top- k sparsification to linear inputs in attention and feed-forward blocks, supports block sparsity, and uses a straight-through

estimator during training (Wang et al., 2024). Spark introduces top- k feed-forward and attention sparsity and evaluates execution (You et al., 2025); recent work also combines sparse model design with inference kernels (Cetin et al., 2026). Our study examines a different set of matched interventions: fixed-threshold gates with their ordinary masked gradients, explicit magnitude pressure, and additional gates on the actual query, key, and value operands. Queries and keys are gated after rotary position encoding. The evidence addresses these design choices in Pythia pretraining; direct comparisons with top- k and structured training are needed to establish their relative advantages.

Task-aware auxiliary optimization. Gradient surgery modifies auxiliary updates when objectives conflict (Yu et al., 2020; Hsieh et al., 2024). We use a task-aware magnitude-pressure rule, denoted OL1, alongside a conventional joint-loss L1 baseline. The OL1 recipe maintains task-only Adam moments, projects conflicting preconditioned pressure updates, and bounds their norm. Its role is to supply a consistent optimization intervention across gate configurations. The matched comparisons evaluate the complete rule; its individual components would require separate ablations.

3 WHAT SPARSITY MAKES AVAILABLE

We consider full-sequence causal inference at sequence length T . For each matrix-product operation o , let N_o count scalar multiplications and let Z_o count those for which at least one operand is exactly zero. We count a multiplication once when both operands are zero. The transformer-block operations are the query/key/value projections, attention output projection, two feed-forward projections, and the attention products $QK^\top$ and PV . The vocabulary projection contributes its full dense count, N_{head} . We define the *model-wide activation sparsity opportunity* as

$$R_{\text{model}} = \frac{\sum_{o \in \text{blocks}} Z_o}{\sum_{o \in \text{blocks}} N_o + N_{\text{head}}}, \quad R_{\text{block}} = \frac{\sum_{o \in \text{blocks}} Z_o}{\sum_{o \in \text{blocks}} N_o}. \quad (1)$$

The reported interventions act on activations; the counter uses the actual operands, including any naturally occurring zeros. Integer counts are pooled over all layers, batches, and validation sequences before division.

This definition connects local zeros to the operations that use them. For example, a zero feed-forward hidden activation affects the following projection. A zero key component can participate in multiple causal query-key pairs. For attention products, we count the union of zero-operand events over valid causal positions using the post-rotation queries and keys. Appendix A.1 gives the counts and denominator explicitly. Layer normalization, softmax, elementwise transformations, data movement, and kernel-launch costs fall outside this multiplication accounting.

Cross-architecture comparisons additionally depend on the block share s_{block} of the counted work:

$$R_{\text{model}} = s_{\text{block}} R_{\text{block}}, \quad s_{\text{block}} = \frac{\sum_{o \in \text{blocks}} N_o}{\sum_{o \in \text{blocks}} N_o + N_{\text{head}}}. \quad (2)$$

This identity helps separate learned block sparsity from architectural differences. Runtime measurements then test how much of the opportunity a particular implementation realizes.

4 A MATCHED INTERVENTION LADDER

We distinguish three choices: where gates create exact zeros, where training penalizes activation magnitudes, and how the optimizer incorporates that pressure. Table 1 names the combinations. The feed-forward hidden state is h . Four-site gating (A4) additionally acts on attention-branch normalization output a , feed-forward-branch normalization output m , and the concatenated attention context z before the output projection. Seven-site gating (A7) adds the post-rotation queries q , post-rotation keys k , and values v .

Gate operators. For $\kappa \geq 0$, the four common sites use a one-sided threshold and the additional attention sites use a sign-preserving threshold:

$$G_{\kappa}^{+}(x) = x \mathbf{1}[x \geq \kappa], \quad G_{\kappa}^{\pm}(x) = x \mathbf{1}[|x| \geq \kappa]. \quad (3)$$

Table 1: Intervention design. The 14M study includes every row. The selected 70M and appendix 410M studies include A0, A1-H, A4+OL1@4, and A7+OL1@7. For local pressure, dose is λ ; for A4/A7, dose is κ with $\lambda = 1$ whenever pressure is present.

Condition	Gated sites	Pressure sites	Update rule
A0	native GELU at h	none	task only
A1-H	ReLU at h	none	task only
A1-H+L1	ReLU at h	h	joint-loss L1
A1-H+OL1	ReLU at h	h	OL1
A4	a, m, h, z	none	task only
A4+OL1@h	a, m, h, z	h	OL1
A4+OL1@4	a, m, h, z	a, m, h, z	OL1
A7	a, m, h, q, k, v, z	none	task only
A7+OL1@7	a, m, h, q, k, v, z	all seven	OL1

At h , the gate replaces GELU. Elsewhere it acts on the named tensor. Gradients pass through retained values. At $\kappa = 0$, the added q, k, v gates are identities in both values and gradients, providing a same-architecture control for the unpressured A4/A7 comparison. The common one-sided gates remain active at this threshold.

Activation pressure. For the tensors captured at the selected layers and sites, the penalty is $\mathcal{P} = J^{-1} \sum_{j=1}^J |A_j|^{-1} \|A_j\|_1$. Thus each tensor contributes its mean magnitude with equal weight. The joint-loss baseline applies AdamW to $\mathcal{L}_{\text{CE}} + \lambda \mathcal{P}$. OL1 instead maintains task-only Adam moments, preconditions the pressure gradient with the task second moment, removes its component opposing the task update when their dot product is negative, and caps the weighted pressure-direction norm at the task-direction norm. Appendix A.2 specifies the update.

What the contrasts isolate. A4 versus A4+OL1@h tests hidden-state pressure under fixed four-site gates. A4+OL1@h versus A4+OL1@4 expands the pressure sites with the gate set fixed. A4 versus A7 adds the three attention-operand gates with pressure absent. Within each of A4 and A7, adding OL1 tests pressure at that configuration’s sites. Comparing these last two effects is descriptive because the pressure set expands together with the gate set. A7 gates with pressure confined to a, m, h, z would complete the fixed-pressure comparison.

The A4+OL1@h cohort was originally labeled more broadly in its run metadata. Artifact inspection established the executed h -only pressure set; we use that operational identity throughout and preserve the original files.

5 EXPERIMENTAL PROTOCOL

Matched pretraining. We train Pythia-configured transformers (Biderman et al., 2023) from random initialization on MiniPile (Kaddour, 2023). Within each size, the conditions share seed-1234 initialization, seed-1234 data order, and the training schedule. The 14M study uses six layers, width 128, and four heads; 70M uses six layers, width 512, and eight heads. All feed-forward widths are four times the hidden width. Each condition receives 712 optimizer steps with a global batch of 1,024 length-2,048 sequences: 1.493 billion scheduled tokens, approximately one training-cache pass. The 14M and 70M peak learning rates are 10^{-3} . AdamW uses $(\beta_1, \beta_2) = (.9, .95)$, $\epsilon = 10^{-8}$, weight decay .1 excluding biases and normalization, norm-1 gradient clipping, one-percent warmup, and cosine decay to ten percent of the peak rate. Parameters and optimizer state remain FP32 during mixed-precision training.

Evaluated conditions. The local 14M pressure coefficients are $\lambda \in \{.05, .1, .5, 1\}$. The A4/A7 gate thresholds are $\kappa \in \{0, .01, .05, .1, .5\}$, with pressure coefficient and OL1 step budget both equal to one. The detailed 14M study contains 35 trained checkpoints. The selected 70M study contains 12: two controls and the two pressured five-threshold recipes. A fixed-budget 410M cohort contributes another 12 checkpoints in Appendix D. These are one-seed comparisons; matched doses characterize sensitivity within one training realization.

Table 2: Matched 14M effects: child minus parent at the same threshold. The first pair of columns is A7 minus A4 without pressure; the other columns add OL1 at each configuration’s respective sites. Positive ΔR means more model-wide opportunity; negative $\Delta \mathcal{L}$ means lower loss. The columns evaluate different interventions under one seed.

κ	Add Q/K/V gates		Add pressure to A4		Add pressure to A7	
	$\Delta \mathcal{L}$	ΔR (pp)	$\Delta \mathcal{L}$	ΔR (pp)	$\Delta \mathcal{L}$	ΔR (pp)
0	-0.0021	+0.006	-0.0122	+0.598	+0.0118	-0.163
0.01	-0.0077	+0.204	-0.0082	+1.173	+0.0170	+0.106
0.05	+0.0038	+0.921	+0.0556	+2.251	+0.0249	+0.736
0.1	+0.0090	+1.472	+0.1286	+2.441	+0.0008	+1.372
0.5	+0.0432	+5.171	+0.3783	+2.498	+0.1265	+12.096

Uniform clipping reference. For the trained A0 and A1-H controls, we calibrate fixed, per-layer/site magnitude thresholds on ten training-cache blocks and apply them only at evaluation to a, m, h, z . A common target quantile $p \in \{0, .1, \dots, .9\}$ controls clipping intensity. Each size therefore has 20 clipping evaluations. This uniform allocation is a reproducible training-free reference; optimized layer allocation and jointly selected site-specific thresholds could improve it.

Common quality and counting pass. We evaluate all 500 MiniPile validation documents after tokenization and concatenation: 338 complete blocks, or 692,224 input tokens, with a 1,444-token tail excluded. Each plotted trained endpoint pairs cross-entropy loss (nats per predicted token) and multiplication counts from the same eager forward pass. All comparisons use this common coverage. Tables report the complete evaluated grids; lines in figures indicate dose order. We use *frontier* for the nondominated evaluated points, where improving opportunity requires accepting higher loss.

6 RESULTS

6.1 EXPANDING PRESSURE IS A CONSEQUENTIAL INTERVENTION

The local 14M comparison establishes that pressure can improve quality and sparsity together in this training regime (Figure 1, left). Relative to the ReLU control, ordinary L1 at $\lambda = 1$ reduces loss from 5.2696 to 5.1023 and raises R_{model} from 2.71% to 3.95%. OL1 produces similar local opportunities: its loss advantage over ordinary L1 at the first three coefficients reverses at $\lambda = 1$. This comparison gives limited support for preferring either complete optimization recipe at the feed-forward hidden state.

The effect changes when pressure reaches more sites (Figure 1, middle). With A4 gates fixed, moving from h -only pressure to four-site pressure raises loss at all five thresholds. At $\kappa = .01$, opportunity is essentially unchanged (8.5858% versus 8.5867%), while loss rises from 5.2045 to 5.4583. At $\kappa = .5$, the expansion adds 2.49 percentage points of opportunity and raises loss by 0.3153. Broader pressure therefore has an identifiable quality cost even when more operations are directly targeted.

6.2 ATTENTION-OPERAND GATES CHANGE THE PRESSURE RESPONSE

Without pressure, adding q, k, v gates raises opportunity increasingly with threshold (Table 2). At $\kappa = .5$, A7 gains 5.17 percentage points over A4 with a loss increase of 0.0432. The $\kappa = 0$ comparison, where the added gates are identities, differs by only 0.006 percentage points and -0.0021 loss, exposing the scale of residual numerical variation between these separately executed controls.

The pressure response differs between the configurations. At $\kappa = .1$, adding seven-site pressure raises opportunity by 1.37 points with a loss change of 0.0008; four-site pressure adds 2.44 points with a loss increase of 0.1286. At $\kappa = .5$, seven-site pressure adds 12.10 points with a 0.1265 loss increase, compared with 2.50 points and 0.3783 for four-site pressure. Low-threshold effects are smaller and mixed. The largest-threshold pressured A7 endpoint has both lower loss and higher opportunity than its pressured A4 counterpart. These results support the combined gate-and-pressure recipe and locate

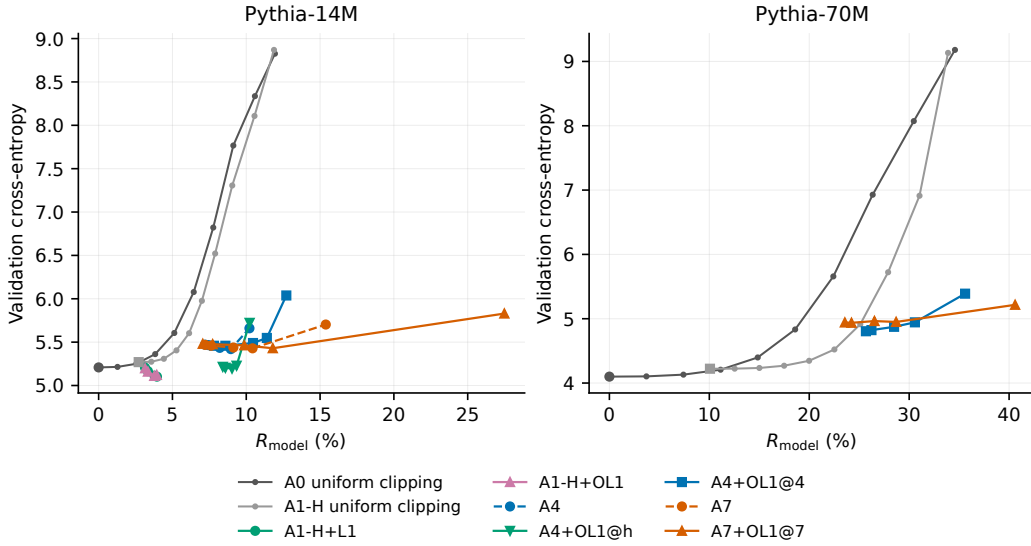


Figure 2: Training and uniform clipping at 14M and 70M. All evaluated clipping quantiles are shown. Curves join tested doses; their nondominated portions define the empirical frontiers. The 14M panel includes the full intervention study, while 70M includes the two selected pressured recipes and controls. Opportunity gains must be read together with the corresponding loss.

Table 3: Maximum evaluated R_{model} (%) within an added cross-entropy allowance δ relative to each size’s A0 control. Trained includes the available training conditions and controls; clipped includes both uniform clipping paths. These are descriptive operating-point selections on one validation set.

Loss allowance	14M trained	14M clipped	70M trained	70M clipped
0.05	9.34	2.55	0.00	7.41
0.10	9.34	4.42	0.00	7.41
0.25	11.80	5.27	10.07	19.99
0.50	15.39	6.13	10.07	22.52
1.00	27.48	6.99	30.57	25.11

the threshold range in which its benefit is most pronounced. Attribution to an interaction at a fixed pressure set requires the additional condition described in Section 4.

6.3 TRAINING EXTENDS THE EVALUATED FRONTIER, WITH QUALITY-DEPENDENT GAINS

Figure 2 compares all main-study training conditions with the complete uniform-clipping paths. At 14M, the trained conditions offer better opportunity at several quality levels. At 70M, the selected training grid reaches larger opportunities, while clipping is stronger near the unmodified model’s quality.

To compare at common quality allowances, Table 3 takes the largest evaluated opportunity among points whose loss is at most $\mathcal{L}_{A0} + \delta$. Selection uses the discrete tested grid. At 14M, training exceeds clipping at each reported allowance. At 70M, clipping reaches 19.99% opportunity at $\delta = .25$, versus 10.07% for the selected trained grid. At $\delta = 1$, training reaches 30.57%, versus 25.11% for clipping. Thus the 70M results support a frontier extension at larger loss allowances; the preferred intervention depends on the quality requirement.

The tighter $\delta = .05$ allowance permits approximately 5.1% higher perplexity. At this allowance, the 14M trained grid reaches 9.34% opportunity versus 2.55% for clipping; at 70M the selected trained interventions exceed the allowance and the control remains the best eligible trained point. The $\delta = 1$ row permits a 2.72 $\times$ perplexity increase and describes a substantially more permissive quality target.

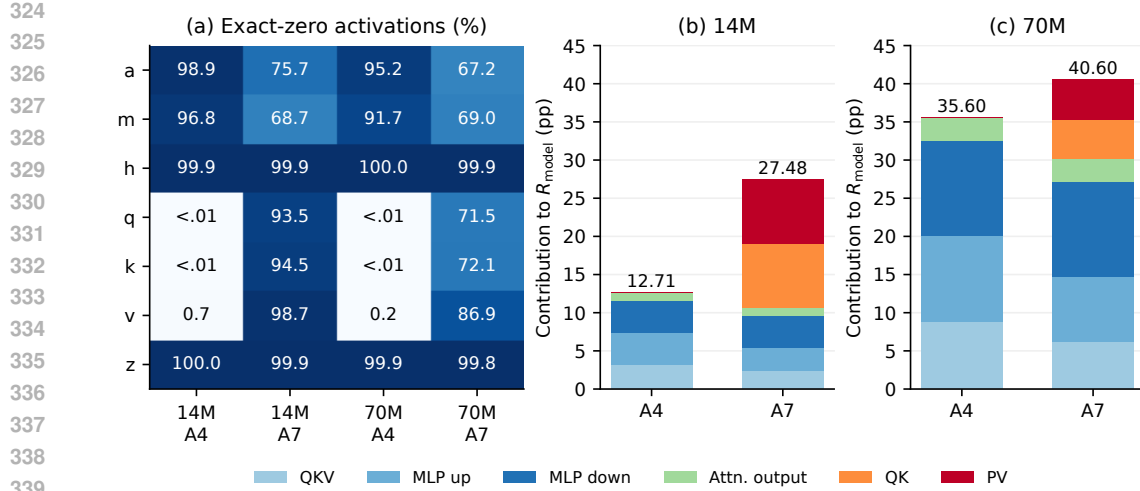


Figure 3: High-threshold ($\kappa = .5$) pressured recipes. Left: pooled exact-zero fractions at the seven activation sites. Right: contributions of each operation to R_{model} , using the complete model denominator. A7 creates substantial attention-product opportunity. Different block/head proportions also affect the totals across sizes.

6.4 WHERE THE ADDED OPPORTUNITY OCCURS

The high-threshold A7 recipe changes the actual attention operands (Figure 3). At 14M, the QK^T and PV products contribute 16.77 percentage points to its 27.48% model-wide opportunity. For the corresponding A4 recipe, these products contribute only 0.057 points to a total of 12.71%. At 70M the attention contribution is 10.42 points for A7 and 0.011 for A4. The operation decomposition therefore explains the additional opportunity opened by gating q, k, v . At these endpoints, h and z are more than 99.8% zero at both sizes. This concentration also motivates a capacity-matched comparison with statically reduced branches. The endpoint measurements describe where zeros occur; intermediate training measurements would be needed to explain how those representations develop.

The total opportunity rises from 27.48% at 14M to 40.60% at 70M for the high-threshold A7 recipe. However, its block-level opportunity falls from 91.75% to 82.15%. The larger model allocates 49.42% of counted work to transformer blocks, compared with 29.95% at 14M (Equation 2). This architectural difference accounts for the higher model-wide fraction despite lower block sparsity. The two sizes establish that the selected recipe can produce substantial opportunity in both architectures; a scaling law would require broader size, data-budget, and seed coverage.

6.5 AN IMPLEMENTATION TEST BOUNDS THE EFFICIENCY CLAIM

We evaluated six 70M checkpoints with a sparse linear path adapted from Cetin et al. (2026) on an H100 NVL. The implementation preserves signed nonzero values, passes shape-level correctness checks, and matches the corresponding dense BF16 validation loss within 10^{-3} . Its full-model scope replaces downstream linear projections and retains dense attention. An official sparse-model positive control achieves a $1.288\times$ speedup.

For our checkpoints, every tested full-model sparse configuration is slower than its dense reference at batch sizes 1 and 32 with $T = 2048$. The best median ratio, dense latency divided by sparse latency, is $0.983\times$ for A4+OL1@4 at $\kappa = .5$ and batch size 1. Separate unfused sparse-attention compositions also remain slower. Appendix E gives all six results, correctness coverage, and implementation limits. The learned zeros expose multiplication opportunity, while execution with this implementation remains slower.

7 DISCUSSION

Implication for sparse model design. The matched branches favor evaluating gate location and optimization together. Pressure on h can improve the local quality–sparsity trade-off, while spreading the same type of pressure to four sites can increase its quality cost. Adding attention-operand gates opens a different source of opportunity and changes the response to pressure at the expanded site set. This motivates architectures and objectives that specify both *which representations should be sparse* and *which operations will consume them*.

From elementwise zeros to executable structure. Our measurements suggest two directions for future interventions. Joint activation and weight pressure could target the union of their zero operands, with overlap explicitly accounted for. Grouped or blockwise activation pressure could encourage channels shared across tokens, improving the regularity available to an implementation. Such constraints introduce a new representation trade-off: shared paths may be easier to execute but can restrict token-specific computation. Testing them requires matched quality, structure, and runtime measurements. The current negative execution result provides a concrete motivation for these extensions.

Scope of the evidence. One seed, one corpus, and a fixed token budget support descriptive matched comparisons. The threshold sweep characterizes sensitivity within this setup; independent seeds would establish how reproducible the effects are. The most direct next controls are A7 gates with four-site pressure and matched Q-Sparse-style top- k and optimized clipping baselines. Downstream tasks, continued training, supervised adaptation, and reinforcement-learning post-training would test whether the observed trade-offs persist beyond this pretraining setting.

The 410M cohort is reported in full in Appendix D, including its different within-family loss response. At a common 1.493-billion-token budget, exposure declines from 106.1 to 21.2 to 3.7 tokens per parameter across 14M, 70M, and 410M. Compute-optimal training research motivates increasing data together with model size (Hoffmann et al., 2022); larger, adequately budgeted comparisons therefore require more resources. The current results leave the cause of the 410M behavior open. Data-budget and optimization controls are needed to distinguish limited training exposure from other explanations.

8 CONCLUSION

Matched interventions show that the location of gates and magnitude pressure substantially changes the quality–sparsity trade-off in transformer pretraining. Attention-operand gating opens opportunity beyond the feed-forward and projection inputs, while broader pressure alone can increase the quality cost. The 14M study and selected 70M results establish a quality-dependent extension of the evaluated uniform-clipping frontier. Operation-level accounting and a negative execution test identify the next requirement: learning sparsity patterns whose structure can support efficient inference.

REPRODUCIBILITY STATEMENT

The appendix specifies the data coverage, model configurations, gate locations, optimization rules, and complete evaluated grids. The accompanying analysis reconstructs figures and tables from immutable evaluation artifacts and records their hashes. Each validation point covers the same 338 complete sequences; all activation and multiplication fractions use pooled integer counts.

AI USE DISCLOSURE

Generative AI assisted with literature search, methodological criticism, implementation and test review, figure code, and drafting. The authors are responsible for checking the experiments, interpretations, citations, and artifacts before submission.

REFERENCES

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron,

Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2304.01373>.

Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, faster, lighter transformer language models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2203.15556>.

Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that scalpel: Improving gradient surgery with an EMA. *arXiv preprint arXiv:2402.02998*, 2024. URL <https://arxiv.org/abs/2402.02998>.

Jean Kaddour. The minipile challenge for data-efficient language models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.

Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-aware thresholding for sparsity in large language models. *arXiv preprint arXiv:2404.08763*, 2024. URL <https://arxiv.org/abs/2404.08763>.

James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-free activation sparsity in large language models. *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2408.14690>.

Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liquan Deng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing law: Towards large language models with greater activation sparsity. *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/luo25i.html>.

Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Orel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU strikes back: Exploiting activation sparsity in large language models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.

Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. Prospare: Introducing and enhancing intrinsic activation sparsity within large language models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.

Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All large language models can be fully sparsely-activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.

Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark transformer: Reactivating sparsity in FFN and attention. *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2506.06644>.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

Zhengyan Zhang, Chaojun Xiao, Qiujieli Qin, Yankai Lin, Zhiyuan Zeng, Xu Han, Zhiyuan Liu, Ruobing Xie, Maosong Sun, and Jie Zhou. Exploring the benefit of activation sparsity in pre-training. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 60040–60056, 2024. URL <https://proceedings.mlr.press/v235/zhang24bq.html>.

A METHODS AND REPRODUCIBILITY

A.1 EXACT ACTIVATION LOCATIONS AND COUNTING

Pythia uses parallel attention and feed-forward residual branches. Their normalization outputs a and m are distinct tensors. Table 4 specifies the operands used by the interventions and counters. The attention output after W_o is a separate diagnostic tensor.

Table 4: Activation locations. B is batch size, H the number of heads, $d_h = d/H$, and d_f the feed-forward width. Pressure captures the selected site’s output after its gate.

Site	Tensor location	Shape	Consuming product
a	attention-branch norm/gate output	$[B, T, d]$	fused QKV
m	feed-forward-branch norm/gate output	$[B, T, d]$	W_1
h	feed-forward activation/gate output	$[B, T, d_f]$	W_2
q, k	immediately after partial RoPE/gate	$[B, H, T, d_h]$	$QK^\top$
v	values from fused QKV/gate	$[B, H, T, d_h]$	PV
z	concatenated context/gate before W_o	$[B, T, d]$	W_o

For one sequence and L layers, the denominator is

$$N_{\text{model}} = L [T(4d^2 + 2dd_f) + dT(T + 1)] + TdV, \quad (4)$$

where V is vocabulary size. The six block counts per layer are $3Td^2$ (QKV), Td^2 (W_o), Tdd_f each (W_1, W_2), and $dT(T + 1)/2$ each ($QK^\top, PV$). For $QK^\top$, a multiplication is zero when $q_{i,r} = 0$ or $k_{j,r} = 0$ with $j \leq i$. For PV , the corresponding event is $p_{i,j} = 0$ or $v_{j,r} = 0$. Future-token pairs are excluded from both numerator and denominator. Projection counters evaluate the actual activation and weight operands. Exact zero means equality to zero in the evaluation dtype. All counts are summed over the 338 sequences before division.

The denominator credits one scalar multiplication per operand pair. Its full-sequence causal convention is held constant across conditions; implementation arithmetic can differ, including fused dense attention. The dense vocabulary head is included in full with zero credited opportunity. Appendix E separately reports the subset of model-wide opportunity covered by the evaluated sparse linear implementation.

A.2 OPTIMIZATION

The joint-loss L1 baseline differentiates $\mathcal{L}_{\text{CE}} + \lambda\mathcal{P}$, clips the combined gradient to global norm one, and performs an AdamW update. OL1 accumulates task and pressure gradients separately across the effective batch. It clips the task gradient to norm one and updates AdamW’s moments using that gradient. With bias-corrected task moments $\hat{m}, \hat{v}$, it forms

$$d_t = \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon_{\text{Adam}}}, \quad d_p = \frac{\nabla_{\theta}\mathcal{P}}{\sqrt{\hat{v}} + \epsilon_{\text{Adam}}}. \quad (5)$$

The inner product and norms below are global over trainable parameters. With $\epsilon = 10^{-12}$, when $\langle d_t, d_p \rangle < 0$ and $\|d_t\|_2^2 > \epsilon$, the pressure direction is replaced by

$$d_s = d_p - \frac{\langle d_t, d_p \rangle}{\|d_t\|_2^2 + \epsilon} d_t; \quad (6)$$

otherwise $d_s = d_p$. With step budget $b = 1$,

$$\rho = \frac{\lambda\|d_s\|_2}{\|d_t\|_2 + \epsilon}, \quad c = \min\left(1, \frac{b}{\rho + \epsilon}\right). \quad (7)$$

After AdamW’s task update, each parameter group receives $-\eta\lambda c d_s$, using its current learning rate η . Weight decay remains part of AdamW. This procedure bounds the auxiliary direction relative to the task direction at the current step. Its geometric constraint leaves finite-step task-loss improvement as an empirical question.

A.3 DATA AND EXECUTION IDENTITIES

The dataset is JeanKaddour/minipile at revision

18ad1b0c701eaa0de03d3cecfdd769cbc70ffbd0.

The tokenizer is EleutherAI/pythia-14m-deduped at revision

7386d9a4ae45aef494a6e704910394def3037fc5.

Encoding adds no special tokens and appends EOS per document. The training cache has 1,491,711,416 tokens and 728,374 complete blocks, with a 1,464-token tail. The 712 full batches schedule 729,088 blocks, so the final batch wraps into the same seeded order for 714 blocks. The validation cache has 693,668 tokens, 338 complete blocks, and the reported 1,444-token excluded tail. Its complete-block coverage is shared by every comparison. The cache SHA-256 identities are:

Training:

da82a2ea2e0080c7fd681c7a93b07d3d9ff3d5357a8640895a82d536a1eaf97c

Validation:

51cd758fda72f14383da30c358a895d0223c0d1d80b31455d2d842c3656d0451

The parameter counts are 14,067,712, 70,426,624, and 405,334,016. The 410M configuration has 24 layers, width 1,024, feed-forward width 4,096, and 16 heads. All configurations use vocabulary size 50,304 and zero hidden/attention dropout. The learning-rate schedule warms up for eight optimizer steps, reaches its peak on step nine, and decays toward one tenth of the peak. Initialization is random from pinned architecture configurations. The 14M cohorts share verified initialization and data-order identities; the larger cohorts load a canonical initialization within size. Training uses FP16 autocast with dynamic loss scaling and FP32 parameters and optimizer states. Different GPU executions can introduce residual numerical variation.

The source artifacts retain terminal training-time validation losses and same-pass eager counting losses separately. We consistently use the latter for quality–opportunity comparisons. This produces small last-decimal differences from earlier draft tables that used terminal losses. The accompanying reduction records source paths and hashes for every trained checkpoint and the upstream clipping summaries. The reduction uses retained evaluation artifacts throughout.

A.4 CLIPPING CALIBRATION AND SELECTION

For each control, the calibration pass uses ten complete training-cache blocks. At every selected layer/site, the target p specifies a quantile of absolute activation magnitude. The resulting fixed thresholds are reused throughout validation. A0 and A1-H have separate calibration and zero-target evaluations. The target p is shared across sites; achieved exact-zero fractions can differ because of the activation distributions and native zeros. Each path is reported in full in Appendix C. The common-loss table uses the best discrete point satisfying the stated allowance relative to that size’s zero-clipped A0 loss. The allowances are descriptive summaries chosen for this analysis; a held-out downstream evaluation would be needed to assess selected operating points independently.

B COMPLETE TRAINED RESULTS AND PAIRED CONTRASTS

Table 5 includes every 14M condition used in the main analysis, including the realized h -only pressure cohort. Dose means λ for A1-H+L1/OL1 and κ for A4/A7. The A4/A7 pressure coefficient is fixed at one. No independent-seed uncertainty is available.

Table 5: Complete 14M training grid. Loss and opportunity are paired in the same complete-validation counting pass.

Intervention	Dose	Loss	$R_{\text{model}} (\%)$
A0	—	5.208573	0.0000
A1-H	—	5.269634	2.7141
A1-H+L1	0.05	5.206144	3.1416
A1-H+L1	0.1	5.165483	3.3363
A1-H+L1	0.5	5.112688	3.7876
A1-H+L1	1	5.102275	3.9493
A1-H+OL1	0.05	5.198063	3.1452
A1-H+OL1	0.1	5.159397	3.3386
A1-H+OL1	0.5	5.110236	3.7680
A1-H+OL1	1	5.121185	3.9384
A4	0	5.470516	7.2120
A4	0.01	5.466488	7.4137
A4	0.05	5.434115	8.2059
A4	0.1	5.419673	8.9530
A4	0.5	5.659678	10.2155
A4+OL1@4	0	5.458276	7.8100
A4+OL1@4	0.01	5.458309	8.5867
A4+OL1@4	0.05	5.489760	10.4564
A4+OL1@4	0.1	5.548254	11.3943
A4+OL1@4	0.5	6.037982	12.7134
A7	0	5.468393	7.2177
A7	0.01	5.458820	7.6176
A7	0.05	5.437875	9.1269
A7	0.1	5.428668	10.4250
A7	0.5	5.702895	15.3868
A7+OL1@7	0	5.480181	7.0542
A7+OL1@7	0.01	5.475801	7.7234
A7+OL1@7	0.05	5.462800	9.8630
A7+OL1@7	0.1	5.429488	11.7968
A7+OL1@7	0.5	5.829407	27.4827
A4+OL1@h	0	5.215749	8.4094
A4+OL1@h	0.01	5.204513	8.5858
A4+OL1@h	0.05	5.195583	9.0356
A4+OL1@h	0.1	5.228699	9.3435
A4+OL1@h	0.5	5.722660	10.2274

Table 6: Complete selected 70M training grid. The 70M study repeats the pressured recipes; it has no unpressured A4/A7 or h -only-pressure branches.

Intervention	Dose	Loss	R_{model} (%)
A4+OL1@4	0	4.805361	25.6725
A4+OL1@4	0.01	4.822850	26.2242
A4+OL1@4	0.05	4.874177	28.4900
A4+OL1@4	0.1	4.943887	30.5743
A4+OL1@4	0.5	5.389480	35.5962
A7+OL1@7	0	4.941206	23.5624
A7+OL1@7	0.01	4.936396	24.2263
A7+OL1@7	0.05	4.963656	26.5239
A7+OL1@7	0.1	4.950079	28.6938
A7+OL1@7	0.5	5.215925	40.6019
A0	–	4.099766	0.0005
A1-H	–	4.222750	10.0658

The different pressure responses can be summarized by

$$D_{\mathcal{L}}(\kappa) = [\mathcal{L}_{A7+OL1@7} - \mathcal{L}_{A7}] - [\mathcal{L}_{A4+OL1@4} - \mathcal{L}_{A4}], \quad (8)$$

$$D_R(\kappa) = [R_{A7+OL1@7} - R_{A7}] - [R_{A4+OL1@4} - R_{A4}]. \quad (9)$$

Table 7 exposes the full dose dependence. At $\kappa = .5$, the difference favors the A7-conditioned recipe on both axes. At the other thresholds the opportunity difference favors the A4 pressure response. Because pressure sites differ, this is a contrast of two configuration-specific treatment effects.

Table 7: Difference between seven-site and four-site pressure effects at 14M. Each row compares the two matched pressure contrasts at one threshold.

κ	$D_{\mathcal{L}}$	D_R (pp)
0	+0.0240	-0.761
0.01	+0.0252	-1.067
0.05	-0.0307	-1.514
0.1	-0.1278	-1.070
0.5	-0.2518	+9.598

Table 8: Architecture and exposure context. The last two columns use A7+OL1@7 at $\kappa = .5$. Block share is fixed by the architecture and full-sequence workload. All conditions use 1.493 billion scheduled tokens.

Model	Parameters	Tok./param.	Block share (%)	R_{block} (%)	R_{model} (%)
14M	14.1M	106.1	29.95	91.75	27.48
70M	70.4M	21.2	49.42	82.15	40.60
410M	405.3M	3.7	87.25	92.40	80.62

C COMPLETE UNIFORM-CLIPPING RESULTS

Each table contains ten common target quantiles for both controls. All rows use the same full validation coverage as the trained endpoints. The complete range includes high-loss points to expose the limits of clipping. The zero-target rows serve as the matched fixed-checkpoint references.

Table 9: Uniform clipping at 14M. p is the calibration target quantile.

p	A0 loss	A0 R_{model} (%)	ReLU loss	ReLU R_{model} (%)
0.0	5.208594	0.0000	5.269633	2.7141
0.1	5.214453	1.2787	5.274073	3.5684
0.2	5.250270	2.5548	5.307940	4.4184
0.3	5.361901	3.8316	5.405056	5.2719
0.4	5.605384	5.1255	5.602965	6.1293
0.5	6.077731	6.4419	5.976274	6.9945
0.6	6.821173	7.7644	6.521474	7.8929
0.7	7.766948	9.1235	7.306175	9.0518
0.8	8.336118	10.5860	8.107618	10.5585
0.9	8.825610	11.9419	8.869450	11.8789

Table 10: Uniform clipping at 70M.

p	A0 loss	A0 R_{model} (%)	ReLU loss	ReLU R_{model} (%)
0.0	4.099766	0.0005	4.222750	10.0658
0.1	4.103608	3.7048	4.223747	12.5370
0.2	4.130721	7.4099	4.234510	15.0099
0.3	4.207335	11.1282	4.269464	17.4855
0.4	4.397795	14.8442	4.346144	19.9851
0.5	4.833057	18.5885	4.522054	22.5156
0.6	5.657785	22.4109	4.914340	25.1077
0.7	6.927989	26.3484	5.722641	27.8935
0.8	8.072236	30.4621	6.912035	31.0351
0.9	9.179397	34.5814	9.133301	33.8912

Table 11: Uniform clipping at 410M.

p	A0 loss	A0 R_{model} (%)	ReLU loss	ReLU R_{model} (%)
0.0	4.547456	0.0110	4.651294	21.0497
0.1	4.553016	7.4632	4.652483	25.9843
0.2	4.581109	14.8833	4.662035	30.9048
0.3	4.672734	22.2428	4.693632	35.8103
0.4	4.931758	29.5388	4.773735	40.7131
0.5	5.585540	36.8043	4.975709	45.6193
0.6	6.721288	44.0981	5.444486	50.5821
0.7	7.758184	51.5636	6.466273	55.6422
0.8	8.311326	60.0129	8.391250	61.7700
0.9	9.027046	70.0471	9.029800	69.1629

D FIXED-BUDGET 410M EXTENSION

The 410M cohort receives the same token exposure and number of updates as the smaller models, with peak learning rate 3×10^{-4} . It extends the two pressured recipes and both controls (Table 13). Its trained families differ from the smaller models: increasing the gate threshold can improve loss while increasing opportunity (Figures 4 and 5). At $\kappa = .5$, A7+OL1@7 has loss 5.1207 and opportunity 80.62%; the A0 control has substantially lower loss, 4.5475. Thus a within-family improvement leaves a considerable gap to the unmodified model.

The exposure is 3.7 tokens per parameter, compared with 21.2 at 70M and 106.1 at 14M. This difference motivates a larger-data control before interpreting the result as a model-size effect. Compute-optimal allocations provide a rationale for scaling the data budget (Hoffmann et al., 2022); they leave the cause of this cohort’s behavior unresolved. The learning-rate screen in Table 12 supports the selected A0 learning rate within its tested grid. The remaining design differences include architecture, exposure per parameter, and optimization trajectory.

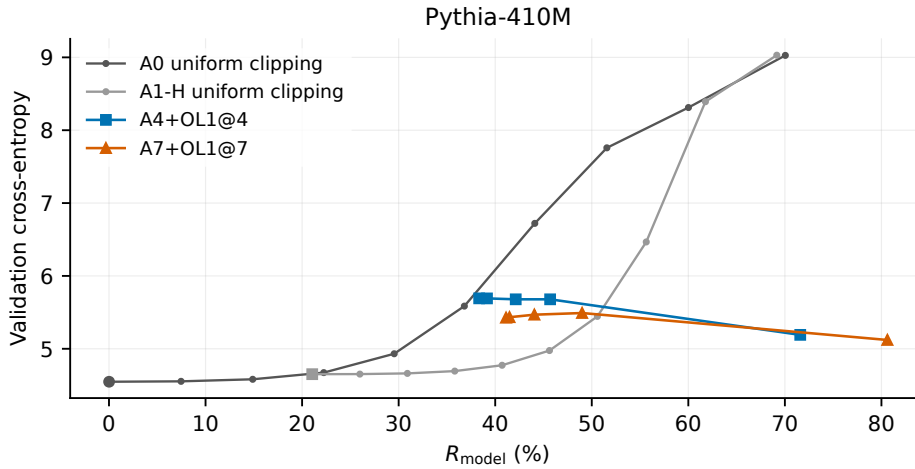


Figure 4: Complete 410M fixed-budget training and clipping paths. The trained threshold response and the quality gap to A0 are both retained.

Table 12: 410M A0 learning-rate screen. Selection used the predeclared mean task loss over steps 649–712. Validation loss is shown as a separate diagnostic.

Peak learning rate	Late training loss	Validation loss	Selected
3e-04	4.493717	4.547456	yes
6e-04	4.513309	4.552068	
1e-03	6.851160	6.786663	

Table 13: Complete selected 410M training grid.

Intervention	Dose	Loss	R_{model} (%)
A4+OL1@4	0	5.692075	38.3387
A4+OL1@4	0.01	5.691362	39.1492
A4+OL1@4	0.05	5.678925	42.1245
A4+OL1@4	0.1	5.678887	45.6880
A4+OL1@4	0.5	5.190966	71.5914
A7+OL1@7	0	5.426296	41.1215
A7+OL1@7	0.01	5.433138	41.4781
A7+OL1@7	0.05	5.467853	44.0532
A7+OL1@7	0.1	5.490749	48.9976
A7+OL1@7	0.5	5.120692	80.6155
A0	–	4.547456	0.0110
A1-H	–	4.651294	21.0497

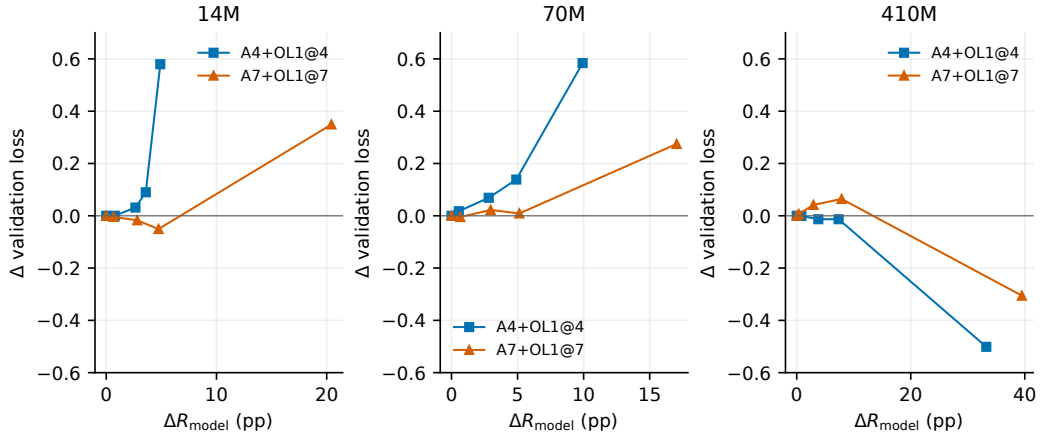


Figure 5: Changes from the zero-threshold point within each pressured family. Lines order increasing gate threshold. The axes show both improvements and deteriorations in validation loss. The 410M response differs from the 14M/70M responses under the common token budget.

E CORRECTNESS AND EXECUTION STUDY AT 70M

The execution study uses six checkpoints: A0, A1-H, and A4+OL1@4/A7+OL1@7 at $\kappa = 0, .5$. The linear implementation is adapted from the sparse-kernel path released with Cetin et al. (2026). Exact signed packing preserves all nonzero values; the original positive-input predicate would discard negative activations in these models. The hardware is an NVIDIA H100 NVL. Full-model comparisons use BF16, sequence length 2,048, batch sizes 1 and 32, and uncached full-sequence prefill with native dense scaled-dot-product attention. They include packing and downstream linear execution, with compilation and CUDA graphs disabled.

For A4/A7, the sparse path covers QKV, W_1 , W_2 , and W_o . For A0/A1-H it covers only W_2 . We report the covered opportunity as $R_{\text{covered}} = \sum_{o \in \text{replaced}} Z_o / N_{\text{model}}$, using the same complete denominator as R_{model} . In particular, the A7 $\kappa = .5$ checkpoint has 40.60% total opportunity but 30.18% covered by these linear replacements.

Table 14: Complete 70M full-model execution results. Batch columns are median paired ratios of dense latency to sparse latency; values above one indicate a speedup. R_{covered} credits only the replaced linear operations. All six sparse paths are slower at both batch sizes.

Condition	R_{model} (%)	R_{covered} (%)	Batch 1	Batch 32
A0	0.00	0.00	0.183×	0.277×
A1-H	10.07	10.07	0.660×	0.618×
A4+OL1@4, 0	25.67	25.67	0.583×	0.277×
A4+OL1@4, .5	35.60	35.59	0.983×	0.707×
A7+OL1@7, 0	23.56	23.56	0.467×	0.252×
A7+OL1@7, .5	40.60	30.18	0.797×	0.405×

Seven paired timing blocks provide each reported full-model ratio. The best case, A4+OL1@4 at $\kappa = .5$ and batch size 1, has median $0.983496\times$ and 10th–90th percentile range $[0.982588, 0.985447]\times$ across timing blocks. This range describes timing dispersion in the measured process. An official SparseLM-0.5B positive control reaches $1.2876\times$, showing that the environment can reproduce a benefit for the source model/path.

Verification covers all seven tested linear shapes and every sentinel checkpoint. Source-checkpoint FP16 validation losses reproduce exactly. The largest full-validation BF16 dense–sparse loss discrepancy is 0.0006854, below the declared 0.001 tolerance. BF16 runtime validation and the canonical counting pass have separate precision provenance; the manuscript’s quality frontiers use the canonical same-pass counting losses.

The study also tests 108 representative linear cases. Both pack-plus-kernel and kernel-only timing remain below parity; the best kernel-only ratio is $0.8359\times$. Separate Q-only or V-only sparse-attention compositions preserve the selected computation but use unfused per-head execution. Their best tested ratio is $0.3612\times$. These checks help locate the limitation beyond full-model packing alone, while leaving open alternative kernels, fused attention, decoding workloads, batching policies, hardware, and explicitly structured training. The result is specific to the implemented path and measured workload.